Inheritance & Constructor

Trainer's Name:
Ms. Shanmuga Priya

Inheritance

```
class Account{}
class Student{..}

class Student extends Account{...} ? No


class SavingsAccount extends Account
{

}
```

Account – Super class
SavingsAccount – Sub class

Animal - class
Dog - Dog is a Animal

Dog is the sub class of Animal

John’s dog = object

Constructor types

Default constructor
user defined default constructor
parameterized constructor

User defined default constructor

```
public Employee()
{
        System.out.println("Inside default constructor");
}
```

Parameterized constructor

```
public Employee(int id, String name, int age) {
        System.out.println("Inside p.c");
        this.id = id;
        this.name = name;
        this.age = age;
    }
```

Method overloading

same method name
- No of arguments
- Order of arguments
- Type of arguments

Creating object for the sub class

```
class Animal{...}
class Dog extends Animal{...}

Dog dog = new Dog();
```

Creates two parts . One is Animal part and other one is Dog part.

Animal part object contains the instance variables of Animal class
Dog part object contains the instance variables of Dog class.

instance variable is maintained in object level
static variable - maintained in class level

Method overriding

Method signature should be same, including the return type.

1) Inheritance
2) Should satisfy method overriding rule
3) Access modifier rule

```
@Override
   void sound()
   {
      System.out.println("Dog barks");
   }
```

Static method inheritance

```
class Animal
{
    static void eat()
    {
        System.out.println("Static Method from Animal");
    }
}
class Dog extends Animal
{
  // @Override - error
    static void eat()
    {
        System.out.println("Static Method from Dog");
    }
}
```

Constructor calling

```
Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    Employee(int id, String name, double salary) {
         this(id, name); - should be the first stmt
         this.salary = salary;
    }
```

Polymorphism

Animal animal = new Dog();

animal.sound();

Using a super class variable, when you store the sub class object reference, and if you call the overridden method, then sub class overridden method will be invoked.

final keyword

final - variable
final - method
final - class

THANK YOU